

PhD Review Briefing

Everything to Know About Article 1 and Article 3

Prepared for the review meeting, Thursday (week of 22 June 2026)

Contents

1	How to use this document	2
2	The thesis spine (say this first)	2
3	Why this matters now: AI and the changing nature of software	3
3.1	The landscape (citable figures, 2025–2026)	3
4	Article 1 — “Do JML Specifications Help LLMs Write Better Unit Tests?”	4
4.1	The question and why it matters	4
4.2	Theoretical background — what counts as a specification	4
4.3	The inference instrument (JML-Inferrer)	5
4.4	Probe-and-backport development methodology	5
4.5	Experimental design (memorise this)	5
4.6	Metrics and statistics	6
4.7	Results in detail	6
4.8	Threats to validity (rehearse these)	7
4.9	Related-work positioning (the “why is this novel” answer)	7
5	Article 3 — “Does Compositional Refinement Strictly Extend Heuristic Specification Inference?”	7
5.1	The question and why it matters	8
5.2	Where the single pass is blind (+ motivating example)	8
5.3	The algorithm (be ready to whiteboard this)	8
5.4	Implementation and the two bugs	9
5.5	Corpus, method, and results	9
5.6	The version-step instance (your thesis link — emphasise this)	10
5.7	Threats to validity (rehearse these)	10
6	Recent literature in your area (2025–2026): the field is hot and your work fits	10
6.1	Test oracles and LLM test generation	11
6.2	LLM-based specification / contract inference	11

6.3	Compositional / modular inference and API evolution (Article 3)	12
7	Known weaknesses — be ready, and fix one before Thursday	12
8	How the two articles connect	13
9	Anticipated panel questions with model answers	14
10	Hostile-examiner playbook	15
10.1	The hardest questions, with honest answers	15
10.2	Traps not to walk into	18
11	Mock-defence drill (rapid-fire)	18
12	Cheat sheet — numbers, glossary, pitches	19
12.1	Article 1 numbers	19
12.2	Article 3 numbers	19
12.3	Glossary (terms the panel may use)	20
12.4	30-second pitches	20

1 How to use this document

This is a deep-prep brief for the review panel meeting — intended to be read end-to-end the night before, with the cheat sheet (Section 12) and the mock-defence drill (Section 11) revisited on the morning of. It is built entirely from the current sources in `journal/article1/` and `journal/article3/`; every number is traceable to those files.

Reading order if you are short on time: Section 2 (thesis spine), Section 7 (must-fix items), Section 11 (rapid-fire defence), Section 12 (numbers). Read Sections 4–5 in full if you want to be able to defend *any* methodological detail.

2 The thesis spine (say this first)

The research programme has two long-term claims, and every paper is a load-bearing piece of one or both:

1. **Adoption.** Formal specifications solve the oracle problem, but nobody writes them because they are expensive (manual specification is $\approx 50\%$ of effort in large verification case studies). If we can *infer* specifications automatically and show they deliver concrete downstream value, we lower the adoption barrier.
2. **Library compatibility via compositional specs.** Inferred specifications act as callee summaries. When a library changes, a change to a callee’s specification implies a change to the specification of every caller that depends on it — so you check the affected call sites instead of re-verifying the whole program.

One-sentence framing: “I infer JML specifications heuristically, and I am demonstrating that those inferred specifications are (a) verifiable, (b) useful to downstream tools, and (c) compositional enough to drive library-compatibility checking.”

The three downstream uses (from Article 1’s introduction) — keep these straight, the panel will probe which is proven and which is future work:

1. Stronger LLM test oracles — *proven in Article 1*.
2. Compositional verification of callers / library compatibility — the *mechanism is Article 3*; the checker itself is future work.
3. Machine-readable contracts for AI coding agents over post-cut-off libraries — *future work*.

The arc between the papers. Article 1 = specs are verifiable and pay off downstream. Article 3 = the inference itself must be *compositional* to be complete, and that compositionality is what makes library-compatibility checking possible. Article 3’s single-pass baseline is exactly Article 1’s `InterproceduralAnalyzer`; Article 3 is the natural follow-up to a limitation implicit in Article 1.

3 Why this matters now: AI and the changing nature of software

This is your “why now” framing — and it is the strongest motivation your thesis has. The more code AI writes, the more acute the oracle problem becomes, because AI generates code *without a statement of what it is supposed to do*. Automatically inferred, machine-checkable specifications are exactly the missing artefact. Lead with this when asked why the work is timely.

3.1 The landscape (citable figures, 2025–2026)

- **Adoption is near-universal.** The Stack Overflow 2025 Developer Survey reports $\approx 84\%$ of developers using or planning to use AI tools, and $\approx 51\%$ of professional developers using them daily. GitHub Copilot reached $\approx 20\text{M}$ users by mid-2025 and is used by $\approx 90\%$ of Fortune 100 companies.
- **A large, fast-growing share of new code is AI-generated.** Microsoft (Nadella) and Google (Pichai) have publicly stated AI writes $\approx 20\text{--}30\%$ of their code, with later Google statements citing higher figures; industry/vendor estimates put AI’s share of all new code in the $40\text{--}46\%$ range. Meta and others have set internal targets for the majority of committed code to be AI-assisted by 2026.
- **But humans are increasingly out of the writing loop, not the *checking* loop.** Only $\approx 30\%$ of AI-suggested code is accepted as-is; developer trust in AI accuracy has *fallen* (Stack Overflow: $\approx 40\%$ to $\approx 29\%$ year-on-year); the single biggest frustration ($\approx 66\%$) is “AI solutions that are almost right, but not quite,” and the second is that debugging AI-generated code is more time-consuming.

The thesis tie-in (say this — it is the whole point)

The reported failure modes of AI-written code — “almost right but not quite,” harder debugging, falling trust, low accept rates — are *the oracle problem at industrial scale*. AI produces plausible code with no specification of intent, so you cannot mechanically tell correct from merely plausible. My work supplies the missing piece three ways: (1) inferred specs are stronger *oracles* for testing AI-written code (Article 1); (2) they are the *callee summaries* compositional verification needs when an AI changes one method in a dependency graph (Article 3); and (3) they are the *machine-readable contracts* an AI coding agent can consume for a library that postdates its training cut-off — which it otherwise has no model of. As more code is machine-written, automatically inferred, verifiable specifications move from convenience to necessity.

Trap warning — do not assert “most code isn’t written by humans”

The literal claim that *most* code is now written by AI is *not* robustly established, and a hostile examiner will demand a source and then dismantle an overstatement. What you can defend: AI *assists* the large majority of developers, and a *large and rapidly growing share of new code* is AI-generated (CEO-reported 25–50%+, vendor-estimated $\approx 40\%$) — but these are mostly self-reported corporate or vendor figures, loosely defined (“written with AI” conflates autocomplete, suggestion, and full generation), and not peer-reviewed measurement. Say “a large and growing fraction of new code is now machine-generated, and adoption is near-universal,” cite Stack Overflow 2025 for adoption, attribute the share figures to their sources as *claims*, and pivot immediately to the oracle-problem consequence — which holds at *any* of these percentages and is the part that matters for your work.

4 Article 1 — “Do JML Specifications Help LLMs Write Better Unit Tests?”

Venue: Journal of Software: Evolution and Process (Wiley/JSEP), research article. **Authors:** Edmonds & Utting. **Status:** submission-ready bar one internal inconsistency (Section 7, #1).

4.1 The question and why it matters

LLMs write plausible unit tests from a signature, but without an oracle the tests assert what the code *does*, not what it *should* do — the *oracle problem*. Konstantinou et al. show empirically that LLM oracles encode actual (often buggy) behaviour. Formal specs are the classical answer but are too costly to write by hand. So the empirical question is practical: *does giving the model an inferred specification measurably improve the tests it writes?* Quality is measured by mutation testing (PIT), the objective dependent variable.

4.2 Theoretical background — what counts as a specification

The specs are organised by the two classical directions of program reasoning; be ready to explain both:

- **Weakest precondition (WP)** (Dijkstra): for statement S and postcondition Q , $WP(S, Q)$ is the weakest entry predicate guaranteeing Q on termination. *Preconditions* are derived this way: a guard-and-throw idiom `if (G) throw` contributes $\neg G$ as a *requires* clause; range checks give two conjuncts; null guards give non-nullness; dereferences contribute definedness conditions.
- **Strongest postcondition (SP)** (Hoare/Cousot): for precondition P , $SP(S, P)$ is the strongest exit predicate. Each `return e` gives `\result == e`; each field `assign this.f = e` gives `this.f == e`.

with field refs wrapped in `\old()`.

- **Key design choice to defend:** branching is emitted as a *guarded conjunction* $(C \Rightarrow r_1) \wedge (\neg C \Rightarrow r_2)$ rather than a monolithic disjunction, so each clause discharges independently. Conditions that merely branch (both sides return) yield *no* precondition — that filter stops internal control flow being mistaken for a contract requirement.
- **Loops** are where syntax alone is incomplete: the WP of a `while` needs an invariant no syntactic procedure can derive. Handled by four heuristics (below).
- **Soundness is recovered *a posteriori*:** the WP/SP framing says *what* to look for, not that each clause is sound; OpenJML ESC discharges every emitted clause and flags failures.

4.3 The inference instrument (JML-Inferer)

- Java 21, $\approx 20,000$ LOC, JavaParser AST, visitor pattern. Three phases: `parse` $\rightarrow$ `infer` (the core) $\rightarrow$ `format` as JML. ≈ 88 ms/file; Apache 2.0 licence.
- **Loop invariants** via four complementary heuristics: bound-based (counter loops), accumulator-pattern detection, monotonicity from update expressions, and bounded symbolic unrolling (3 iterations) as fallback; a conservative weak invariant if all fail. Non-trivial-invariant rate 85.3% over 68 loops.
- **Interprocedural propagation:** two passes — analyse in dependency order and cache each spec, then propagate across call sites by formal-to-actual substitution. Standard-library calls resolve against a curated spec database (`String`, `Math`, `List`, `Map`); unknown calls are uninterpreted (conservative). *This single-pass propagation is exactly what Article 3 shows is incomplete.*
- **Soundness gating:** heuristics unsound under two’s-complement (e.g. `\result  $\geq 0$` for `x * x` or `Math.abs` on `int/long`) are gated by return type.
- **Verification stack:** forked OpenJML emitting SMT-LIB `define-fun-rec` for `\sum`, `\product`, `\num.of` (upstream reports these “not yet supported”). Strictness fixed at four flags: `--code-math=safe`, `--spec-math=bigint`, `--arithmetic-failure=hard`, `--nullable-by-default`. Default solver `z3 4.13.4` (4.7.1 and 4.16.0 selectable). `cvc5` available but not default (non-termination on the preamble).

4.4 Probe-and-backport development methodology

This is a genuine methodological contribution; expect questions on what makes it different from “just ask an LLM.” Four steps, applied per unrecognised pattern:

1. *Specify by example* — reduce the failing pattern to a minimal method; LLM (or author) drafts candidate JML.
2. *Verify with the SMT oracle* — OpenJML ESC discharges or rejects each clause under the fixed strictness; the verifier is a *non-negotiable filter* (a clause the LLM finds plausible but Z3 rejects never enters the corpus).
3. *Encode as a probe test* — the minimal method + discharged contract is pinned in the regression suite (576 tests at submission).
4. *Backport via an agentic coding assistant* — the agent extends the right analyser until the probe passes with no regression (strictness fixed, so every regression is real).

Headline of the loop: reduced the end-to-end suite from 188 failing to 86 of 576 (pass rate 67.3% → 85.1%). The verifier-in-the-loop is the load-bearing element distinguishing it from direct LLM inference (which shows only 72.4% run-to-run consistency). *Note:* this is a dev-tool workflow, distinct from LLM-at-inference-time (which is deferred).

4.5 Experimental design (memorise this)

Controlled four-phase comparison on **312 methods** from Apache Commons Lang 3.14 (11 classes), holding the model fixed and varying *only* the input:

- **P1** — signature alone (baseline).
- **P2** — signature + a *method-agnostic* edge-case checklist (same block for every method: nulls, empty/min/max collections, numeric boundaries, FP specials, unicode, overflow combos).
- **P3** — signature + inferred JML specification (the treatment).
- **P4** — full source code (upper-bound baseline).

Why P2 exists: it is the control for the alternative hypothesis that *any* structured guidance, not specifications specifically, raises mutation score. If a generic checklist matched P3, the spec contribution would be indistinguishable from any structured prompt.

Worked example (`MutableInt.add(Number)`): P1 = bare signature; P3 adds `requires operand != null; assignable this.value; ensures this.value == \old(this.value) + operand.intValue();`; P4 = the two-line body. Signature held constant across phases.

LLM config: Gemini 2.0, temperature 0.3, max tokens 4,096, top-*p* 0.95, **5 runs** per method per phase = 6,240 generations (≈25,000 incl. pilot tuning).

4.6 Metrics and statistics

- **Spec quality:** OpenJML discharge rate; coverage rate; strength (strong = both sides non-trivial, partial = one, weak = neither).
- **Test quality:** test count, compile rate, pass rate, mutation score via **PIT 1.15.3**, **DEFAULTS** operator group (conditionals-boundary, increments, invert-negs, math, negate-conditionals, return-vals, void-calls, and the *-returns set), 10 s per-test timeout. Residual equivalent-mutant rate estimated 4.7% (Papadakis sampling, dual-rater, $\kappa = 0.83$).
- **Statistics:** mean/SD/median/IQR + 95% bootstrap CIs (10,000 iters); paired *t*-tests (Wilcoxon signed-rank corroboration for non-normal samples); Cohen’s *d* effect size; **Bonferroni** across 24 contrasts (6 pairwise × 4 metrics), per-test $\alpha' \approx 0.0021$. Post-hoc power > 0.999 on the P1-vs-P3 mutation contrast (min detectable effect 2.2 pp; observed 40.7 pp).
- **Categorisation:** six categories extending Dragan et al.’s stereotypes; inter-rater $\kappa = 0.87$.

4.7 Results in detail

Metric (mean per method)	P1	P2	P3	P4
Tests generated	9.0	38.0	33.5	54.7
Mutation score (%)	27.5	81.8	68.2	94.8
Compile rate (%)	89.2	87.8	91.5	93.2
Pass rate (%)	94.1	92.3	96.8	97.1

- Test count: **+272%** P3 over P1 (95% CI [245, 299]).
- Mutation: **+40.7 pp** P3 over P1 ($68.2 - 27.5$; $p < 0.001$, Cohen’s $d = 2.41$, CI [2.08, 2.79]).
- P3 closes $\approx 60\%$ of the sig-to-source gap ($(68.2 - 27.5)/(94.8 - 27.5)$) in $\approx$ an order of magnitude smaller input (5–12 lines JML vs. 30–150 lines of body). Residual 26.6 pp gap to P4 is because source effectively hands over the oracle.
- **Category gains (P1→P3):** Control +47.1, Utility +40.6, State +36.6, Factory +19.5, Acc/Mut +19.1 pp. Looping methods with a non-trivial invariant score 72.4% vs. 54.1% without.
- **Engine:** 99.0% coverage (309/312; Jdoctor 45.2%); 100% determinism vs. 72.4% direct-LLM; OpenJML discharges 92.9% of 985 clauses (pre 94.9%, post 85.9%, loop-inv 81.1%, assignable 100%).

4.8 Threats to validity (rehearse these)

- *Internal:* sampling non-determinism (5 runs + low temp); prompt confounding (minimal prompts, P4 control). Spec oracle is the verifier, not human judgement — so numbers mean “consistent with the implementation,” not “matches documented intent.”
- *External:* single library (Commons Lang), mature/well-structured so may flatter the approach; *other* category (callbacks/IO) has 0 methods; single LLM (Gemini 2.0, budget reason); Java-only. Defence: P1→P3 gain (27.5→68.2) is inconsistent with mere training-set recall (else P1 would already $\approx$ P4); literature reports consistent qualitative ordering across GPT/Claude/Gemini.
- *Construct:* mutation-testing limits (equivalent mutants, operator representativeness); spec oracle = implementation, so for contracts weaker than the code (**sort stability for duplicates** is the canonical case) the inferred spec is sound-but-over-specified.
- *Conclusion:* Bonferroni-corrected; power > 0.999 .
- **Three categorical non-discharge limits** (the 7.1% residue — know these cold): (i) recursive multiplicative growth (factorial, Fibonacci, power — emitted with literal small-input bounds like `n <= 12`); (ii) quantified invariants over mutable arrays (`arr[i] *= factor` — Z3 array-theory, 240 s timeout); (iii) OpenJML for-each translation hides `val == arr[count]`. These are SMT/JML-translation limits, *not* inferer unsoundness.

4.9 Related-work positioning (the “why is this novel” answer)

- *vs. Houdini:* Houdini guesses from fixed templates and prunes with ESC/Java; we replace the template guess with WP/SP-organised structural pattern matching and extend the oracle to OpenJML with quantifier support.

- *vs. Daikon / PreInfer*: dynamic, need test suites/execution traces and give “likely” invariants; we are purely static (any compilable Java, no tests).
- *vs. Toradocu / Jdoctor*: documentation-based; effective only where Javadoc is thorough. Complementary — Jdoctor finds 25 exception preconditions we miss; we find 37 it misses.
- *vs. direct LLM inference (SpecGen, Endres, Teuber&Beckert)*: we are deterministic (100% vs. 72.4% consistency) and verifier-gated.
- *vs. TOGA/TOGLL, EvoSuite/Randoop/Korat*: those *consume* specs (or learn oracles from corpora); we *infer* the specs they need — the complementary upstream half of the pipeline.
- *Calibration point*: Molinelli et al. report LLM oracles at 43% mutation vs. 45% human; Schäfer et al. report 70% compile rate (we raise to 91.5%).

5 Article 3 — “Does Compositional Refinement Strictly Extend Heuristic Specification Inference?”

Venue: targeting **ICSE 2027** (IEEEtran conference, double-blind); SANER is the realistic fallback (body is SANER-sized, tight for ICSE’s 10-page limit). **Type**: measurement study.

5.1 The question and why it matters

Most inferers fold in cross-method information by *single-pass propagation*: at each call site substitute the callee’s cached preconditions and append them. The question: *is that single pass already enough, or does a second, top-down WP refinement pass discover preconditions the single pass cannot?* If single-pass suffices, the second pass is wasted engineering; if not, downstream consumers get a richer spec.

5.2 Where the single pass is blind (+ motivating example)

Single-pass is correct for unconditional callee requirements but structurally blind to two shapes:

- **Branch-conditional obligations.** A call `n(args)` guarded by `if (g)` means the caller’s obligation is `g ==> p[args]`, not `p[args]`. Single-pass either appends unconditionally (over-constrains) or skips (misses it) — both wrong.
- **Polymorphic-dispatch obligations.** For `x.n()` with multiple implementations, the sound caller precondition is the *disjunction* `(p_1 || ... || p_k)` over candidates — single-pass resolves one nominal callee and cannot produce it.

Motivating example: `scale` calls `safeDivide(value, factor)` (which requires `divisor != 0`) only inside `if (factor != 0)`. Appending `factor != 0` would forbid the `scale(value, 0)` call the method explicitly handles; skipping loses the obligation. The compositional pass lifts through the guard to the correct branch-conditional clause. A non-tautological corpus case: a call requiring a non-empty list guarded by a size check yields `size > 0 ==> list.size() > 0`.

5.3 The algorithm (be ready to whiteboard this)

Inputs: a populated `SpecificationCache`, the call graph, and a signature→AST map. Mutates the cache in place.

Driver. Iterate strongly-connected components (SCCs) of the call graph in *reverse topological order* so callees refine before callers.

- Singleton SCC (no recursion): refine once. On an acyclic graph a single reverse-topological sweep suffices — each callee precedes its callers, so no clause it gains can invalidate an already-visited caller.
- Cyclic SCC (mutual recursion): refine each method, repeat to fixpoint (cap 16 iterations). Terminates because each method’s clause set only grows and is bounded by the finitely many substituted callee clauses; the cap only guards pathological nesting, not non-termination.

Per-method refinement — for each `MethodCallExpr` in the body:

1. *Resolve the callee* by simple-name match (no full type harness — a documented measurement limitation).
2. *Substitute arguments* (regex with word boundaries so `formal s` doesn’t clobber `this.size`); reject side-effecting actuals (`++`, `--`) as unsound.
3. *Lift through enclosing guards* — accumulate the conjunction of `if`/ternary conditions containing the call; emit `(guard) ==> (substituted); else branch contributes !(guard);` side-effecting guards rejected.
4. *Detect polymorphic dispatch* via the class-hierarchy index; emit the disjunction over candidates (sound: at least one contract must hold for dispatch to be safe).
5. *Conjoin* the lifted clause into the caller’s set if not present.

Termination-aware postcondition propagation: the callee’s postconditions propagate into the caller *only if* the callee terminates (a flag the inferrer maintains) — a non-terminating callee’s postcondition holds *only if* it returns.

What it is / is not: a *lifting scaffold* over the single-pass result, not a full WP transformer. It does *not* compute WP through arithmetic/array statements, handle aliasing, lift through loops, or run SMT — each is handled elsewhere in the pipeline.

5.4 Implementation and the two bugs

≈410-line `CompositionalAnalyzer`, opt-in, depends only on the existing model + `JavaParser`. A 15-test unit suite passed; the corpus-scale run surfaced two real bugs — both a `lastIndexOf('.')` that took the rightmost dot (which falls inside dotted parameter types like `java.lang.String`) in the polymorphic-candidate lookup and the call-site resolver. Use this as a *methodological strength*: “the corpus run finds bugs the unit suite misses.”

5.5 Corpus, method, and results

Corpus: five libraries, four idiom families — Commons Lang 3.14.0 (3,494 methods), Commons IO 2.13.0 (2,111), Commons Math 3.6.1 (6,645), jOOL 0.9.14 (3,691), Vavr 0.10.4 (5,111). Total **1,712 files, 21,052 methods**. Lang+IO match the predecessor work (no corpus confound). Harness: `parse` → build call graph → run isolated inference (snapshot baseline) → `refineAll()` → diff and categorise each new clause.

	Lang	IO	Math	jOOL	Vavr
Methods in cache	3,494	2,111	6,645	3,691	5,111
Methods refined	1,513	750	2,417	862	1,728
% of cache	43.3	35.5	36.4	23.4	33.8
New precondition. clauses	5,755	1,983	27,240	7,309	3,453
avg / refined method	5.06	3.86	14.47	9.52	2.82

- **RQ1:** strictly extends on *every* library — refines 23–43% of methods (median 35.53%), adds **45,740** preconditions and 24,895 postconditions. Single-pass not subsuming anywhere. Means are modest (not driven by a few huge methods; maxima 710/216/208/177/41 are deep-tail outliers).
- **RQ2 (shape distribution):** branch-conditional ($\Rightarrow$) dominant on 4/5 libraries (Lang 57.8, IO 45.9, Math 67.1, Vavr 56.2%); jOOL is the exception (null checks 69.8%, branch-conditional still 26.0%). Disjunctive (polymorphic) on every library, up to 9.8% (Vavr). Both shapes structurally outside single-pass reach.
- **RQ3:** 34 s total (1.62 ms/method); no SCC hit the fixpoint cap (call graphs acyclic under signature-matching). Per-library refine ms: Lang 2,164, IO 664, Math 13,414, jOOL 2,791, Vavr 15,039.
- **Downstream probe (preliminary):** P3 (isolated spec) vs. P3C (refined spec) into gemini-2.5-flash, 11-class Commons Lang subset, temp 0.3, 5 runs. Mean test count +9.1% (8 extractable classes, 6/8 favour P3C); biggest gains on validation-heavy classes (Validate +54.8%, BooleanUtils +27.5%); one regression (CharUtils −23%). No mutation score; tests didn’t compile. *Report as non-zero signal only.*

5.6 The version-step instance (your thesis link — emphasise this)

Commons Lang 3.12.0 → 3.14.0, diffing inferred specs over the 3,188 methods common to both: **525 changed** (16.5%), of which **144 have strengthened preconditions**, 87 weakened, 90 mixed (plus postcondition changes). This is exactly the “candidate-breaking set” a behavioural- compatibility checker consumes — callee changes propagating into caller specifications on a real two-year version step. Spot check: a mix of real tightenings (added non-null guards on CharRange, CharSet, Builder fields) and inferer noise; full release-note validation is a separate study, but the set is the right size/shape to make that tractable.

5.7 Threats to validity (rehearse these)

- *Internal:* deterministic (byte-identical reruns); effect-size deltas not significance tests (statistics inapplicable in a deterministic regime); runtime is single-platform (OpenJDK 21, G1GC, Windows 11) — clause counts are platform-independent, wall-clock is not.
- *Construct:* “refined” = gained ≥ 1 clause by exact string equality, no normalisation — so the new-clause count is an *upper bound* on genuinely-new logical obligations ($a < b$ vs. $b > a$ both counted). Shape buckets are regex-based; *other* is 0.2–1.7%.
- *Callee resolution:* simple-name match; arity check rejects mismatches; arity-pass rate 53% (Vavr) to 75% (IO). Unresolved sites would either *add* clauses (strengthening the result) or be rejected (no effect) — so the reported numbers are a *lower bound* on true compositional gain. (Strong rebuttal point.)
- *External:* 5 libraries chosen to span idioms; **Guava excluded** (call-graph stack-overflow fixed, but source-jar size blew the parsing budget — harness engineering, not analyser correctness; algorithm has no per-library coupling); baseline is one specific single-pass implementation.

6 Recent literature in your area (2025–2026): the field is hot and your work fits

Use this section to project confidence. The honest, positive headline is: *your area is one of the most active in software engineering right now, it is being validated industrially, and recent independent results corroborate the premises your papers rest on — while the specific niche you occupy (deterministic, verifier-gated static inference feeding LLMs, plus compositional refinement for library compatibility) is still largely unclaimed.*

Three positive talking points to lead with

(1) Timely & industrial. Mutation-guided LLM test generation is now deployed at Meta (Automated Compliance Hardening, 2025); the oracle problem has a dedicated ASE 2025 track-level study. Your RQ is exactly where the field’s attention is. **(2) Independently corroborated.** A leakage-controlled ASE 2025 study (13,866 oracles, 135 post-2024 Java projects) finds LLM oracles reach *human-comparable* mutation scores — and a 2026 study finds *prompt/context matters more than model choice*. Both directly support your central claim that *what you put in the prompt* (a specification) is the lever, which is the whole point of $P1 < P2 < P3$. **(3) Distinctive.** The 2025–2026 spec-inference wave (SpecGen, VeriAct, AutoReSpec, counterexample-guided modular specs) is overwhelmingly *LLM-sampled with iterative verifier feedback*; your engine is *deterministic and verifier-gated by construction* — a different and defensible point in the design space.

6.1 Test oracles and LLM test generation

- **“Do LLMs Generate Useful Test Oracles? An Empirical Study with an Unbiased Dataset”** (Di Grazia, Ernst et al., ASE 2025). 13,866 oracles, 135 projects, all tests post-2024-09-01 to avoid training leakage; LLM oracles reach mutation scores comparable to human-written ones. *You already cite this (your molinelli2025oracles); it is your calibration point (43% vs 45%). Engaging the current state-of-the-art head-on is a strength.*
- **“Understanding LLM-Driven Test Oracle Generation”** (arXiv 2601.05542, 2026). Finds prompting strategy has a stronger effect than model choice. *Direct support for your input-varying design and your single-model defence — the lever is the prompt content, not the model.*
- **“Do LLMs generate test oracles that capture actual or expected behaviour?”** (Konstantinou et al., 2024–25). LLM oracles tend to encode *actual* (often buggy) behaviour. *You already cite this — it is the motivation for supplying an external specification.*
- **LLM unit-test-generation survey** (arXiv 2511.21382, 2025) and **Meta ACH / mutation testing** (Engineering at Meta 2025; your foster2025meta). *Evidence the problem is survey-worthy and industrially deployed.*
- **“Evaluating LLM-Based Test Generation Under Software Evolution”** (arXiv 2603.23443, 2026). *New, and squarely supports Article 3’s version-step framing — the evolution angle is a live, independently-pursued topic, not a niche you invented.*

6.2 LLM-based specification / contract inference

- **“Beyond Postconditions: Can LLMs infer Formal Contracts...”** (Richter & Wehrheim, arXiv 2510.12702, 2025) and **“Next Steps in LLM-Supported Java Verification”** (Teuber & Beckert, arXiv 2502.01573, 2025). *Both already cited — the closest JML neighbours; your deterministic, OpenJML-discharged engine contrasts with their sampled approaches.*

- **SpecGen** (your `ma2024specgen`) — iterative verifier-feedback JML synthesis, 72.5% of 385 programs. *The natural baseline; your angle is determinism + the probe-and-backport development loop.*
- **New 2026 preprints worth a sentence each:** “Automatic Generation of Formal Specification and Verification Annotations Using LLMs and Test Oracles” (arXiv 2601.12845); “VeriAct: Agentic Synthesis of Correct and Complete Formal Specifications” (arXiv 2604.00280); “AutoReSpec” (arXiv 2604.03758); “Integrating Symbolic Execution with LLMs for Program Specifications” (arXiv 2506.09550). *All sampled/agentic — cite to show currency and to position your verifier-gated determinism as the contrast.*

6.3 Compositional / modular inference and API evolution (Article 3)

- **“Counterexample-Guided Inference of Modular Specifications”** (PACMPL/OOPSLA 2025, doi 10.1145/3720505). *The strongest recent neighbour for Article 3 — inferring the callee specs modular verification needs. Contrast: theirs is CEGIS/solver-driven; yours is a lightweight WP-lifting scaffold over a heuristic baseline, measured at corpus scale.*
- **“Compositional Inductive Invariant Inference via Assume-Guarantee Reasoning”** (arXiv 2509.06250, 2025). *Reinforces that “compositional beats global” is a current, accepted finding — backs your RQ1 motivation.*
- **“Understanding the Impact of APIs’ Behavioral Breaking Changes on Client Applications”** (Proc. ACM SE / FSE, doi 10.1145/3643782) and the Maven-Central breaking-changes study. *Establish that behavioural (not just syntactic) breaking changes are a recognised, under-tooled problem — precisely the gap your version-step artefact addresses.*

How to deploy this in the room. If asked “is this still novel given how fast LLMs move?”, answer: “The field is moving fast *toward* my premises — 2025–26 work independently confirms that input/specification matters more than model and that LLM oracles can match humans. What remains unfilled is a *deterministic, verifier-gated* inference engine and a *compositional refinement* step that produces the candidate-breaking set for API-compatibility checking. Recent modular-spec and behavioural-breaking- change papers show both halves are live problems others are circling but not solving the way I do.” (*Action: 2–3 of the new 2026 preprints are worth adding to the related-work sections before submission to show currency; the already-cited ASE 2025 / Richter / Teuber / Meta references keep you on the current frontier.*)

7 Known weaknesses — be ready, and fix one before Thursday

#1 (FIX BEFORE MEETING) — Article 1 table contradicts its own narrative

The abstract, intro, and results prose all state “P2 generates the most tests but achieves a *lower* mutation score than P3,” and the whole “test count is a poor proxy for oracle strength” argument depends on it. But Table 4 (`results_journal.tex`) reports mean mutation score **P2 = 81.8%** vs. **P3 = 68.2%** — P2 *higher* than P3 by 13.6 pp — and every per-class row shows $P2 > P3$. The data table says the opposite of the prose. If a panellist opens the table you cannot defend the narrative. **Action:** decide which is correct (are the P2/P3 mutation columns swapped, or is the narrative wrong?), reconcile every mention — abstract, intro, the “Why P2 produces more tests than P3 yet scores lower” paragraph, and the summary — and rebuild the PDF. Highest-priority item.

#2 — Article 3: “you measured the input, not the output”

Article 3 proves the second pass *adds clauses*, not that they help any downstream tool. The LLM probe is +9.1% mean test count, $n = 11$, high variance, one -23% regression, no mutation score, tests didn’t compile. **Defence:** the contribution is the *structural prerequisite* — the richer clauses provably exist on five libraries — which is necessary for any downstream gain; the downstream study is explicitly future work. Don’t over-claim the probe.

#3 — Article 3: most of the gain may not need two passes

Branch-conditional implications are 46–67% of additions, but the discussion concedes a single pass *could* recover them by lifting through the guard during its existing AST walk. That leaves the genuinely two-pass-only contribution (polymorphic disjunctions) at 1.8–9.8%. **Defence:** the categorical argument is that single-pass *by construction* cannot produce disjunction over dispatch candidates from a sibling-cache lookup; that holds against *any* single-pass implementation. Frame the branch-conditional half as “recoverable but not recovered by the deployed baseline.”

#4 — Article 3: Guava is excluded

Predecessor papers used Guava; Article 3 drops it (source-jar size). Reads as “doesn’t scale” / “didn’t hold.” **Defence:** harness engineering, not analyser correctness; the algorithm has no per-library coupling. **Best fix:** an overnight run with more heap closes it definitively — flagged as the single highest-value improvement in the red-team.

#5 — Article 3: single version step, single library

The compatibility framing (525 changed specs) rests on one library, one two-year delta. **Defence:** the structural claim is the five-library measurement; the version step is an intentional single instance. A second diff (e.g. Commons IO 2.11 vs 2.13) is ~ 1 hour and defuses it.

#6 — Article 1: scope and generality

Single library, single model (Gemini 2.0), mutation-score-as-quality; the *other* category has 0 methods. **Defence:** Commons Lang is a standard SE-research subject; P4 is the upper bound; P1→P3 gain rules out training-set recall; planned follow-up is service-boundary code where the oracle problem is most acute.

Lower-severity, one-liner each: regex shape buckets with an unanalysed “other” (Art. 3); “414-line” is LOC not complexity; polymorphic disjunction candidate-set sizes not shown (2–3 informative vs. 20+ vacuous); “dominate on four of five” is uneven (jOOL has it only third) — soften to “substantial ($>30\%$) on every library”; sort-stability / underdetermined- by-code limit (Art. 1, own it as a known structural boundary).

8 How the two articles connect

- **Shared engine.** Both use JML-Inferer. Article 1’s single-pass `InterproceduralAnalyzer` is precisely the baseline Article 3 shows is not subsuming.
- **Shared corpus anchor.** Commons Lang (+IO) appear in both, so cross-paper comparison is confound-free.

- **Three downstream uses, divided across papers** (see Section 2): use 1 proven in Art. 1, use 2 mechanism in Art. 3, use 3 future work.
- **The narrative arc.** Art. 1: specs are verifiable and pay off downstream. Art. 3: inference must be compositional to be complete, and that compositionality enables library-compatibility checking.

9 Anticipated panel questions with model answers

Q. What is the actual contribution beyond “LLMs + specs”?

(1) First effect-sized, mutation-tested evidence that *inferred* (not hand-written) specs improve LLM test quality, with P2 isolating that it’s the specification, not generic prompting; (2) a verifiable, deterministic inference engine (92.9% discharge, 99% coverage); (3) the probe-and-backport methodology; and in Art. 3, evidence that inference must be compositional, plus the mechanism for library-compatibility checking.

Q. Why mutation score, not coverage or bug count?

Coverage rewards executing code; the oracle problem is about *asserting* the right thing. Mutation score is the standard proxy for assertion strength (injected faults the tests catch). P2 vs. P3 is the proof that test count and coverage would mislead — 100% coverage can coexist with 4% mutation score.

Q. The specs are heuristic and incomplete — why trust them?

We don’t ask for trust; we verify. OpenJML ESC discharges 92.9% of clauses against the implementation; flagged clauses (3.8%) go to a review queue and are excluded from P3 prompts; unsupported clauses (3.4%) are three categorically identified SMT-tractability limits, not unsound inference.

Q. Isn’t single-pass propagation already standard and enough?

That is Art. 3’s question; the answer is no on all five libraries — 45,740 new preconditions, 23–43% of methods refined. Two shapes (branch-conditional, polymorphic disjunction) are structurally beyond single-pass reach, and the reported gain is a *lower bound* (53–75% arity-pass).

Q. Does Article 3 disprove the thesis? (it does not — be crisp)

It confirms it. The thesis needs compositionality to be non-redundant; Art. 3 shows single-pass is not subsuming anywhere, and the version step shows callee-spec changes propagate into caller specs — the exact signal a compatibility checker consumes. The open question (future work) is whether the richer specs *also* improve downstream outcomes, not whether compositionality matters.

Q. How is probe-and-backport different from prompting an LLM for JML?

The verifier-in-the-loop. Direct LLM inference shows 72.4% run-to-run consistency and proposes clauses Z3 rejects. In probe-and-backport, OpenJML is a non-negotiable filter: only discharged clauses become probe tests, and the inferrer is extended deterministically until those probes pass. The output is a deterministic engine, not a sampled one.

Q. Why only Gemini 2.0 / why not multiple models?

Budget: 6,240 generations per full run, $\approx 25,000$ incl. pilots. The defended contribution is the *ordering* $P1 < P2 < P3 < P4$, not absolute scores; the literature reports consistent qualitative orderings across GPT/Claude/Gemini. Multi-model replication is parameterised in the package and named as a limitation.

Q. ICSE or SANER for Article 3?

ICSE 2027 is the target (A*, strongest placement). Body is SANER-sized; plan is to tighten to ICSE's 10-page limit, with SANER as fallback if the Guava run and a second version step don't land in time.

Q. What is the through-line to submission / what's next?

Art. 1: reconcile the P2/P3 inconsistency, then submission-ready for JSEP (replication package, CRediT, ORCIDs, data statement all present). Art. 3: Guava inclusion + a second version-step diff + a corpus-scale mutation re-run move it from “structural prerequisite” to “downstream-impact.” Longer term: the agentic-programming use case and a compatibility checker built on the version-step artefact.

10 Hostile-examiner playbook

You have flagged that one panellist is hostile. Treat that as a known input, not a threat: hostile questions are usually *predictable*, and a calm, prepared answer to a sharp question is the single most credibility-building thing you can do in the room. The goal is not to “win” against the person — it is to show the rest of the panel that you know exactly where your own work is strong and where it is bounded.

How to handle the person (not just the questions)

- **Separate the attack from the argument.** Answer the technical point; ignore the tone. Never mirror hostility.
- **Concede fast and precisely when they are right.** “Yes, that is a real limitation — here is exactly how far it extends, and here is why it does not sink the contribution.” Conceding a true point *costs you nothing* and removes their momentum. Over-defending a weak point is what loses panels.
- **Bridge.** “That is a fair challenge. The way I would frame it is...” — acknowledge, then redirect to what you *did* show.
- **Know your boundary line.** For every weakness, have the one-sentence statement of *what you claim* vs *what you do not claim*. Hostile examiners win by making you defend a claim you never actually made.
- **Slow down.** Pause, restate the question in your own words (“So the concern is whether...”), then answer. This buys thinking time and stops you answering a harder question than was asked.
- **Don't bluff.** If you don't know, say “I haven't measured that; my expectation is X because Y, and it's a clean follow-up.” A fabricated answer to a hostile expert is fatal; an honest boundary is not.

10.1 The hardest questions, with honest answers

Q. State your thesis contribution in one sentence. (The classic opener — have it memorised verbatim.)

“I show that JML specifications can be inferred automatically by deterministic, verifier-gated static analysis, that those inferred specifications measurably improve a concrete downstream task (LLM test generation), and that the inference must be compositional — which both completes the specifications and yields the artefact a library-compatibility checker consumes.” Say it the same way every time.

Q. Did your heuristics overfit? Were any of the 312 evaluation methods in your probe/development corpus? (The most dangerous question — know the exact answer before Thursday.)

This is the attack to pre-empt. The probe-and-backport loop *tuned* the heuristics on examples; if those examples overlap the 312-method evaluation set, the 92.9% discharge and the P3 gain are partly measuring fit to your own training data. **You must walk in able to state precisely:** (a) the probe corpus is minimal, hand-reduced example methods, *not* Commons Lang methods [confirm this is true]; (b) whether any Commons Lang method seeded a probe. If there is clean separation, say so plainly — it neutralises the attack. If there is partial overlap, concede it and fall back to: the heuristics are *structural patterns* (null-guard, range-check, accumulator), not method-specific rules, so generalisation is to the pattern, not the instance; and the cross-library result in Article 3 (21,052 methods, four idiom families the heuristics were never tuned on) is your generalisation evidence. *Action: verify the probe/eval separation and write the one-line answer now.*

Q. Your 312 methods come from 11 classes — they are not independent. Isn’t this pseudoreplication, and aren’t your confidence intervals too narrow?

Concede the statistical point honestly: methods within a class share structure, so treating all 312 as independent overstates the effective sample size, and a linear mixed-effects model with class as a random effect would be the textbook analysis. Then redirect on magnitude: the design is *paired* (P1 vs P3 on the same method), which removes method-level variance; and the effect is so large (40.7 pp, $d > 2$) that no plausible clustering correction reverses the direction or significance — clustering widens intervals, it does not flip a 40-point gap. Offer the mixed-effects re-run as a cheap robustness check for the camera-ready. *Do not pretend the independence assumption was correct.*

Q. Cohen’s d above 2 (up to 3.07) is implausibly large for empirical SE. Isn’t that a red flag for a confound?

It is large, and you should not act surprised by the question. The reason it is large is structural, not artefactual: P1 (signature only) has *no oracle at all*, so its mutation score floor (27.5%) reflects near-absent assertions, while P3 supplies explicit postconditions — the contrast is closer to “oracle vs no oracle” than to two competing techniques, which is exactly when huge effect sizes are expected. The honest framing: the effect size is large *because the baseline is weak by construction*, and that is why the more informative comparison in the paper is P3-vs-P4 (input density), not P3-vs-P1 (absolute). Concede that d -against-a-null-baseline is not the headline a careful reader should anchor on.

Q. OpenJML discharge checks the spec against the same code you inferred it from. That’s circular — you’ve proven nothing about correctness.

Agree with the precise version of the claim and reject the overstatement. *Agree*: discharge is a soundness/consistency check (“no clause is inconsistent with the implementation”), not a completeness or intent-matching check — you say this explicitly in the threats section, and the canonical counterexample (sort stability for duplicate keys) is one you raise yourself. *Reject*: “proven nothing” is wrong — consistency is a real and non-trivial property (it catches the 3.8% flagged clauses that were syntactically valid but semantically false, e.g. integer-overflow boundaries), and it eliminates rater subjectivity. The boundary line: “I claim the inferred clauses are sound against the implementation and useful as oracles; I do not claim they recover documented intent — that is the underdetermined-by-code limitation I state.”

Q. Better mutation score just means the tests kill more mutants. Did the specification-guided tests find any real bugs?

Concede the gap directly: mutation score is the standard *proxy* for oracle strength, and you measure the proxy, not real-fault detection on a curated bug set (e.g. Defects4J). Redirect: the proxy is the field’s accepted instrument (PIT, Papadakis survey), and the literature shows 100% coverage can coexist with 4% mutation score, so mutation score is precisely the metric that distinguishes behavioural assertions from coverage padding — which is the distinction your P2-vs-P3 result turns on. A Defects4J real-fault study is the clean, honest follow-up, and you should name it as such rather than claim bug-finding you didn’t measure.

Q. This looks like three thin papers sliced from one project (you cite your own predecessor work). Where is the thesis-level contribution?

Don’t get defensive about self-citation — a cumulative thesis is normal. Frame the arc as one argument in three movements: (1) inference is possible and verifiable [predecessor]; (2) it delivers measurable downstream value [Article 1]; (3) it must be compositional, and that compositionality is what enables the library-compatibility application [Article 3]. The unifying contribution is not any single paper — it is the claim that *deterministic, verifier-gated, compositional spec inference is a viable foundation for both LLM tooling and compatibility checking*. Each paper is a necessary premise; remove any one and the thesis claim is unsupported.

Q. You used LLMs and an agentic coding assistant to build and evaluate the tool. What is your own intellectual contribution?

Answer it head-on; in 2026 this is a fair and increasingly common question. The intellectual contribution is the *method and the framing*, not the keystrokes: the WP/SP organisation of the heuristics, the *verifier-as-non-negotiable-filter* design (the insight that the LLM’s plausibility judgement is insufficient and must be gated by discharge), the experimental design that isolates specifications from generic prompting (P2), and the compositional-refinement formulation. The LLM is an instrument inside a loop *you* designed; the determinism result (100% vs 72.4%) is precisely the evidence that the engine is not “an LLM in a trench coat.”

Q. Article 3’s analyser is 414 lines. That’s a weekend script, not a research contribution.

Reject the LOC-equals-contribution premise calmly. The contribution is the *measurement and the structural claim*, not the artefact size: that a deployed single-pass inferer is provably non-subsuming on 21,052 methods across four idiom families, with a categorical explanation (two clause shapes unreachable by construction). A small, auditable pass is a *virtue* for a measurement study — it isolates the variable. Add: the run also surfaced two real bugs the unit suite missed, which is itself a methodological point about corpus-scale evaluation.

Q. You disabled the tests that failed (for-each, array-mutation). Isn’t that hiding negative results?

Own it as documented scientific bookkeeping, not concealment. The disabled cases are categorically characterised (for-each translation, Z3 array-theory timeouts) and reported *as limitations in the paper*, with the reason each fails — they are solver/translation tractability limits, not inferer unsoundness, and you state which is which. The distinction that protects you: you disabled tests for *known external-tool limits you documented*, you did not delete tests because the inferer was wrong. If anything it strengthens the honesty of the discharge numbers, which exclude rather than fudge them.

Q. Show me the 86/576 number. Is the raw log in the replication package?

Be precise and unflustered: the 86/576 endpoint is author-validated, the suite is reproducible via a single Maven invocation on JDK 21, and the figure is suite-composition-dependent (it is not a clean cross-time metric, so don’t let them bait you into plotting it against differently-sized historical suites). *Action before Thursday: confirm the raw submission-run log is in the replication package or be ready to say exactly where it is and regenerate it on request.* Not being able to produce data on demand is what a hostile examiner is fishing for.

Q. Is this computer science or just software engineering plumbing?

Don’t be provoked. The science is the empirical claims and the structural arguments: a controlled, effect-sized result that specification *content* (not prompt verbosity) drives oracle strength; a categorical proof that single-pass propagation cannot express two clause shapes; a corpus-scale measurement of how often that gap bites. The tool is the instrument that makes those claims testable — the same relationship empirical SE has always had to its artefacts.

10.2 Traps not to walk into

- **Don’t over-claim correctness.** Every time you are tempted to say the specs are “correct”, say “sound against the implementation”. The gap between those is the underdetermined-by-code limit and you should reach it first, not be pushed to it.
- **Don’t defend the P2/P3 inconsistency on the fly.** If they open Table 4, do not improvise — say “that is a transcription error I am correcting; the intended claim is X” (after you have decided which way it goes, weakness #1). Fix it before the meeting so this never arises.
- **Don’t claim multi-model generality.** You ran one model. Claim the *ordering*, not absolute scores, and cite the cross-model corroboration in the literature.
- **Don’t let “compositional refinement is necessary” be overstated.** You showed the deployed single pass is non-subsuming; you did *not* show no single pass could ever close the branch-conditional half. Hold that line (weakness #3).

- **Don't accept “you measured the input not the output” as fatal.** It is the scoped claim, by design — structural prerequisite, downstream impact is future work (weakness #2).

11 Mock-defence drill (rapid-fire)

Cover the right column and answer from memory.

Prompt	One-line answer
P3 over P1, test count?	+272% (CI 245–299).
P3 over P1, mutation?	+40.7 pp ($d = 2.41$).
P4 absolute mutation?	94.8%; P3 closes $\approx 60\%$ of the gap.
Why is P2 in the design?	Control: isolates spec vs. generic guidance.
Discharge rate?	92.9% of 985 clauses.
Coverage vs. Jdoctor?	99.0% vs. 45.2%.
Determinism vs. LLM?	100% vs. 72.4% consistency.
Probe-suite progress?	188 $\rightarrow$ 86 of 576 (67.3 $\rightarrow$ 85.1%).
Three non-discharge limits?	recursive mult.; array-mutation invariants; for-each translation.
Underdetermined-by-code case?	sort stability for duplicate keys.
Statistics correction?	Bonferroni, 24 contrasts, $\alpha' \approx 0.0021$.
Art. 3 RQ1 result?	Strictly extends on all 5 libs; +45,740 preconditions.
% methods refined?	23–43% (median 35.53%).
Dominant shape?	branch-conditional, 46–67% (4/5 libs).
Polymorphic share?	1.8–9.8% (every lib; peak Vavr).
Two shapes single-pass can't do?	guard-lifted implications; dispatch disjunctions.
Runtime?	34 s total, 1.62 ms/method; no fixpoint firing.
Why lower-bound result?	arity-pass 53–75%; unresolved sites only add or reject.
Version-step numbers?	525/3,188 changed; 144 strengthened preconditions.
Why Guava excluded?	parsing budget (harness), not analyser correctness.
Two bugs found at scale?	<code>lastIndexOf(' .')</code> on dotted param types $\times 2$.

12 Cheat sheet — numbers, glossary, pitches

12.1 Article 1 numbers

- Corpus 312 methods, Commons Lang 3.14, 11 classes; 4 phases; 5 runs; 6,240 generations.
- Tests +272% (CI 245–299). Mutation P1 27.5 / P2 81.8 / P3 68.2 / P4 94.8 (**weakness #1**); +40.7 pp, $d = 2.41$.
- $\approx 60\%$ of sig-to-source gap; 5–12 vs 30–150 lines.
- Engine 99.0% coverage; 92.9% discharge (985 clauses); 100% vs 72.4%.

- Categories: Control +47.1, Utility +40.6, State +36.6, Factory +19.5, Acc/Mut +19.1.
- Config: Gemini 2.0, temp 0.3, top- p 0.95, 4096 tokens; PIT 1.15.3.

12.2 Article 3 numbers

- 5 libraries, 1,712 files, 21,052 methods.
- Refined 23–43% (median 35.53%); +45,740 pre, +24,895 post.
- Shapes: branch-conditional 46–67% (4/5); disjunctive 1.8–9.8% (all).
- Cost 34 s, 1.62 ms/method; no fixpoint; arity-pass 53–75%.
- Version step: Lang 3.12→3.14, 525/3,188 changed, 144 strengthened.
- Downstream probe: +9.1% mean test count (preliminary, $n = 11$).

12.3 Glossary (terms the panel may use)

- **WP/SP** — weakest precondition / strongest postcondition; the backward/forward predicate transformers organising inference.
- **Discharge** — OpenJML ESC verifying a clause against the implementation; soundness check, not completeness.
- **Mutation score** — % of injected faults (mutants) the test suite kills; proxy for oracle strength.
- **SCC** — strongly-connected component of the call graph; mutual recursion forms a cyclic SCC requiring fixpoint iteration.
- **Branch-conditional clause** — `guard ==> obligation`; the dominant Art. 3 addition.
- **Polymorphic-dispatch disjunction** — `(p_1 || ... || p_k)` over candidate callee contracts.
- **Probe-and-backport** — draft JML → verify → pin as probe test → agent extends inferrer until it passes.

12.4 30-second pitches

Article 1: “Heuristically inferred JML specifications make LLM-generated unit tests substantially better — 272% more tests and 40.7 mutation points over signatures alone — and a control phase shows it’s the specification doing the work, not generic prompting. The specs are produced automatically and 93% verify under OpenJML.”

Article 3: “The standard way to fold callee contracts into a caller — single-pass propagation — is not enough. A top-down WP refinement pass adds 45,740 new preconditions across five libraries, in shapes single-pass structurally cannot produce. And on a real version step those changes propagate, giving the candidate-breaking set a compatibility checker needs.”

Built from the current sources in [journal/article1/](#) and [journal/article3/](#). The discrepancy in weakness #1 reflects the source as it stands today and should be resolved before the meeting.